Requirement Analysis - Anime World Shop

Project: E-Commerce Mobile Application (SETEC Institute, Year 4 Mobile Application, SU25 Final Project) **Product:** Anime World Shop - a mobile shop for anime figures, plush, merchandise and trading card games (TCG) **Version:** 1.0 - 16 October 2026

Related documents: Use Case Diagram and Specifications - ERD - Database schema - API documentation

---

1. Introduction

1.1 Purpose

This document describes what Anime World Shop must do (functional requirements), how well it must do it (non-functional requirements) and the business rules it follows. It is the reference for the design, the database, the API and the tests, and every requirement is traced back to the project brief in section 10.

1.2 Problem statement

Anime collectors in Cambodia mostly buy figures and cards through Facebook pages and Telegram chats. Customers have to ask for the price and stock of every item, send payment screenshots by hand, and have no way to follow their order. Shop owners keep stock and orders in chat history and spreadsheets.

1.3 Product scope

Anime World Shop is made of three parts that share one database:

Part	Who uses it	Built with
Mobile app - customer side	Shoppers	Flutter (Android, iOS, web)
Mobile app - admin side	Shop owner and staff (same app, opened by an admin account)	Flutter
Backend REST API + web admin panel	The mobile app; staff on a computer (`/admin`)	Python Flask, MySQL

In scope: accounts, catalogue, search and filters, cart, wishlist, checkout, KHQR and cash on delivery payment, order tracking, receipts, notifications, contact messages, settings, and the shop management tools. Out of scope for version 1.0: delivery-company integration, refunds through the app, multiple shops (marketplace), loyalty point redemption, and product reviews written by customers (the `reviews` table exists; see section 11).

1.4 Definitions

Term	Meaning
KHQR	Cambodia's national QR payment standard (Bakong). Any Cambodian bank app can pay a KHQR code.
COD	Cash on delivery

Term	Meaning
Series	The anime a product belongs to, e.g. One Piece, Jujutsu Kaisen, Pokemon
TCG	Trading card game (booster boxes, structure decks)
JWT	JSON Web Token, the login token the app sends with every request
OTP / reset code	The 6-digit code emailed for "Forgot password"
Must / Should / Could	Priority of a requirement (MoSCoW)

---

2. Stakeholders and users

Stakeholder	Role	Main needs
Customer	Buys products	Find items fast, see real stock and prices, pay easily, know where the order is
Visitor	Has not signed in yet	Create an account, sign in, recover a forgotten password
Admin (shop owner / staff)	Runs the shop	Handle new orders quickly, keep stock right, add products, answer customers
Bakong KHQR (external system)	Payment network	Receives the QR payment request and reports when it is paid
Email service (external system)	Sends mail	Delivers password reset codes
Lecturer / assessor	Grades the project	Brief coverage, documentation, working demo
Development team	Builds and maintains the system	Clear requirements, tests, readable code

The actors and their use cases are drawn in the Use Case Diagram.

---

3. Technology selection

The brief lets each team choose its backend technology. Our choices and the reasons:

Layer	Choice	Why
Mobile app	**Flutter 3 / Dart**	Required by the brief; one code base for Android, iOS and web
State management	**Provider**	Simple, official recommendation, easy for the whole team to read
HTTP client	**Dio**	Interceptors add the login token and turn errors into one exception type
Backend	**Python Flask** (Blueprints)	Small and readable; easy to build a JSON API
ORM	**SQLAlchemy**	Models in Python, same code for MySQL and the SQLite test database
Database	**MySQL 8**	Relational data with foreign keys fits orders, items and stock; matches the ERD
Security	**Flask-JWT-Extended**, **bcrypt**	Stateless login tokens; passwords stored as salted hashes
Payment	**bakong-khqr** package	Generates real KHQR codes and checks their payment

Layer	Choice	Why
		status
API documentation	**OpenAPI 3 + Swagger UI**	Interactive docs at `/api/docs`, plus a generated Markdown copy
Web admin	**Flask-Admin + Flask-Login**	Back-office on a computer with no extra front-end project
Localisation	**flutter_localizations / gen-l10n**	English and Khmer
Source control	**Git + GitHub**	Required by the brief; one branch per feature

---

4. Functional requirements

Priority: **M** = Must (in the brief), **S** = Should (needed for a real shop), **C** = Could (bonus). Status: all requirements below are implemented in version 1.0 unless marked *Planned*.

4.1 User management (FR-U)

ID	Requirement	Priority
FR-U01	The app shows a **splash screen** with the logo while it checks for a saved login, then opens the shop (customer), the admin area (admin) or the login screen.	M
FR-U02	A visitor can **register** with full name, email, password and an optional phone number.	M
FR-U03	A user can **log in** with email and password and stays logged in until they log out.	M
FR-U04	A user who **forgot the password** can request a 6-digit code by email, enter it, and choose a new password.	M
FR-U05	A customer can view their **profile**: photo, name, email, phone, order counts by status and member level.	M
FR-U06	A customer can **edit the profile** (name, phone, profile photo).	M
FR-U07	A logged-in user can **change the password** after entering the current one.	S
FR-U08	A customer can add, edit, delete and choose a default **delivery address**.	S
FR-U09	A user can **log out**; the app asks for confirmation first.	M
FR-U10	Accounts have a **role** (customer or admin); admin-only screens and API calls are refused for customers.	M

4.2 Product browsing (FR-P)

ID	Requirement	Priority
FR-P01	The **home screen** shows promotion banners, anime series tabs, featured products and new arrivals.	M
FR-P02	The customer can browse **product categories** with the number of products in each.	M
FR-P03	A **product list** shows photo, name, price, sale price and stock, as a grid or a list.	M
FR-P04	**Product details** show all photos, description, brand, series, category, price, sale price and stock, with Add to cart and wishlist buttons.	M
FR-P05	The customer can **search** by product name, brand or series, with suggestions while typing and recent searches.	M
FR-P06	The customer can **filter** by category, series, brand (several at once), price range and "in stock only", and **sort** by newest, price (low-high, high-low) or name.	M
FR-P07	Hidden products (`is_active = false`) never appear in the shop.	S

4.3 Cart and wishlist (FR-C)

ID	Requirement	Priority
FR-C01	The customer can **add a product to the cart** from the list or details screen, with a visible confirmation.	M
FR-C02	The customer can **update the quantity** of a cart line, up to the available stock.	M
FR-C03	The customer can **remove a product** from the cart, or empty the whole cart after confirming.	M
FR-C04	The cart shows subtotal, delivery fee, discount and total, and how much more is needed for free delivery.	S
FR-C05	The customer can **apply a coupon code** in the cart; invalid or expired codes show the reason.	S
FR-C06	The customer can **add and remove wishlist (favourite) products**, view them as swipe cards, a board or a selectable list, and move selected items to the cart.	M

4.4 Orders and payment (FR-O)

ID	Requirement	Priority
FR-O01	**Checkout** lets the customer choose the delivery address, delivery method (standard delivery or store pickup) and payment method (KHQR or cash on delivery), and add a note.	M
FR-O02	Placing the order checks stock again, creates the order and its items, lowers stock and empties the cart.	M
FR-O03	An **order confirmation** screen shows the order number, total and next steps.	M
FR-O04	For KHQR orders the app shows a **KHQR code** with a countdown; the app detects the payment automatically and shows a receipt.	C (bonus: online payment)
FR-O05	**Order history** lists the customer's orders, current ones first, with status and photos.	M
FR-O06	**Order details** show a status tracker (placed, paid, shipped, delivered), items, address, payment and totals.	M
FR-O07	The customer can **confirm an order was received**, **buy the same items again**, and **view, save or share the receipt** of a paid order.	S

4.5 Additional features (FR-A)

ID	Requirement	Priority
FR-A01	**Notifications** tell the customer when an order is placed, paid, confirmed, shipped, delivered or cancelled, and when the shop answers a message; unread ones are counted on the bell icon and can be marked read or deleted.	M
FR-A02	**Settings** let the user choose light, dark or system theme and English or Khmer, and save which kinds of notification they want (orders, offers, news).	M
FR-A03	An **About** screen shows the shop, the app version and credits; **Privacy policy** and **Terms of use** pages are available.	M
FR-A04	A **Contact** screen shows the shop's Telegram, phone, email, address and hours, and lets the customer send a message about an order, payment, product or anything else.	M
FR-A05	A **Help** screen answers common questions.	S

4.6 Shop administration (FR-AD)

ID	Requirement	Priority
FR-AD01	An admin account opens an **admin dashboard**: total revenue, revenue for the last 7 days, orders to handle, low stock, open messages and a live activity feed with one-tap actions.	S

ID	Requirement	Priority
	The dashboard refreshes every 20 seconds.	
FR-AD02	The admin can **search and filter orders** by status and move an order to its next step (confirm, ship, deliver) or cancel it. The customer is notified each time.	M
FR-AD03	The admin can **add, edit, hide and delete products**: name, category, series, brand, price, sale price, stock, featured, description and photos.	M
FR-AD04	The admin can **add, rename and delete categories**.	M
FR-AD05	The admin can **view customers** with their order count, total spent, addresses and orders, and **block or unblock** a customer.	M
FR-AD06	The admin can **read and answer contact messages**; the answer reaches the customer as a notification.	S
FR-AD07	The same management tools are available in a **web admin panel** at `/admin` for use on a computer.	C

---

5. Backend and API requirements

5.1 Core functions (from the brief)

ID	Core function	How the backend provides it
BE-01	User authentication and authorization	`/api/auth/*` (register, login, me, forgot password, verify code, reset password); JWT on every protected route; role check on `/api/admin/*`; blocked users refused
BE-02	Product management	Public `/api/products`, `/api/products/<id>`, search suggestions; admin `/api/admin/products` create, update, delete, photos, stock
BE-03	Category management	Public `/api/categories`; admin `/api/admin/categories` create, rename, delete
BE-04	Customer management	Own profile `/api/profile`, addresses `/api/addresses`, preferences; admin `/api/admin/customers` list, details, block
BE-05	Shopping cart management	`/api/cart` add, update quantity, remove, clear, coupon; wishlist `/api/wishlist`
BE-06	Order management	`/api/checkout`, `/api/orders` place, list, details, received, KHQR payment; admin `/api/admin/orders` list, details, status
BE-07	API for the mobile app	64 REST endpoints, JSON in and out, documented (section 5.2)

5.2 API requirements

ID	Requirement
API-01	All requests and responses use **JSON** (photo uploads use multipart form data).
API-02	Every response has the same envelope: `{"success": true/false, "message": "...", "data": ...}`; errors add `"errors"`.
API-03	**Validation** happens on the server for every input; a `422` response lists each wrong field, e.g. `{"email": "Email is not valid"}`.
API-04	Errors use the right HTTP status: `400` bad request, `401` not logged in, `403` not allowed or blocked, `404` not found, `409` conflict (e.g. not enough stock), `422` validation, `429` too many attempts, `500` unexpected error (still JSON).
API-05	Lists are **paginated** (`page`, `per_page`, `total`, `total_pages`).
API-06	The API is **documented** with OpenAPI 3: interactive Swagger UI at `/api/docs` and `docs/API.md`; an automated test fails if any route is not documented.

---

6. Non-functional requirements

ID	Category	Requirement	How it is met / verified
NFR-01	Usability	Modern, clean, consistent UI; a new user can find a product and check out without help.	One design system (Roboto, sky-blue brand colours, rounded cards); every screen designed in Figma first; empty, loading and error states on every list
NFR-02	Usability	Clear feedback for every action.	Snack bars, toasts, confirmation dialogs for destructive actions (red button), field errors under inputs
NFR-03	Accessibility	Text and icons readable in light and dark mode; touch targets at least 40 px.	Dark theme with its own colour scheme; Material icons with tooltips
NFR-04	Localisation	The app works in English and Khmer and remembers the choice.	gen-l10n `.arb` files; customer screens translated (admin tools in English for v1.0)
NFR-05	Performance	Screens show data within about 2 seconds on a normal connection; lists load in pages; photos are cached.	Pagination (max 50 per page), photos cached on the device with `cached_network_image`
NFR-06	Security	Passwords are never stored in plain text.	bcrypt hashes
NFR-07	Security	Only logged-in users reach their own data; only admins reach admin data.	JWT (7-day expiry), ownership checks on every order/address/cart query, role check, blocked accounts rejected
NFR-08	Security	Password reset cannot be brute-forced.	Code valid 10 minutes, locked after 5 wrong tries, 60 s between requests, max 5 requests per hour
NFR-09	Security	Secrets are not in the source code.	`.env` file (git-ignored) for database URL, secret keys and Bakong token
NFR-10	Reliability	Stock can never go below zero and an order is all-or-nothing.	Stock re-checked when the order is placed, in one database transaction
NFR-11	Reliability	Unpaid KHQR orders do not hold stock forever.	Cancelled and restocked after 15 minutes
NFR-12	Maintainability	Code is organised and tested.	Flask Blueprints, Flutter screens/services/providers folders; 87 backend tests (pytest) and 169 Flutter tests; `flutter analyze` with no issues
NFR-13	Portability	Runs on Android, iOS and the web.	Flutter; API base URL in one constant
NFR-14	Compatibility	The backend works with MySQL in use and SQLite in tests.	SQLAlchemy models; `flask init-db` creates the tables

---

7. Business rules

ID	Rule
BR-01	An email address can register only once. Passwords have at least 6 characters.
BR-02	Delivery is free when the subtotal is $50 or more; otherwise standard delivery costs $2. Store pickup is always free.
BR-03	A cart quantity cannot exceed the product's stock; sold-out or hidden products cannot be ordered.

ID	Rule
BR-04	A coupon is valid only if it is active, not expired, and the subtotal reaches its minimum. Percentage coupons can have a maximum discount.
BR-05	Order numbers are unique and readable, e.g. `AWS-20261016-0001`.
BR-06	Orders move only forward: pending -> confirmed -> shipped -> delivered. Pending and confirmed orders can be cancelled; shipped and delivered orders cannot.
BR-07	An unpaid KHQR order cannot be confirmed; it waits for payment or is cancelled after 15 minutes, and its stock is returned.
BR-08	Cancelling an order returns its items to stock.
BR-09	A category can be deleted only when it has no products.
BR-10	A blocked customer cannot log in or use an existing token until an admin unblocks them.
BR-11	A product's sale price must be lower than its normal price; stock is 0 or more.
BR-12	Each status change and each answer to a message creates a notification for the customer.

---

8. Data requirements

The system stores 16 entities. The nine suggested in the brief are all present:

Brief entity	Table	Extra tables we needed
Users	`users`	`password_resets` - reset codes
Categories	`categories`	`series` - anime franchise of a product
Products	`products`	`coupons` - promo codes
Product Images	`product_images`	`payments` - KHQR payments
Cart	`cart_items`	`notifications`
Orders	`orders`	`contact_messages`
Order Items	`order_items`	`reviews` - prepared for ratings
Addresses	`addresses`	
Favorites / Wishlist	`favorites`	

Order items copy the product name and unit price at the time of purchase, so old orders stay correct after a price change. Full diagram: ERD (PNG, PDF); full definitions: schema.sql.

---

9. Screen inventory

The brief asks for at least 20 screens. The app has **41** (31 customer, 10 admin).

#	Screen	Brief item	#	Screen	Brief item
1	Splash	Splash Screen	22	My Orders	Order History
2	Login	Login	23	Order Details	Order Details
3	Register	Register	24	Receipt	(extra)
4	Forgot Password	Forgot Password	25	Notifications	Notifications
5	Verify Code	Forgot Password	26	Settings	Settings
6	Reset Password	Forgot Password	27	My Addresses	(extra)
7	Password Changed	Forgot Password	28	Help	(extra)

#	Screen	Brief item	#	Screen	Brief item
8	Profile	User Profile	29	Contact Us	Contact Information
9	Edit Profile	Edit Profile	30	About	About Application
10	Change Password	(extra)	31	Privacy Policy / Terms	(extra)
11	Home	Home Screen	32	Admin Dashboard	(admin)
12	Categories	Product Categories	33	Admin Orders	(admin)
13	Product List	Product List	34	Admin Order Details	(admin)
14	Product Details	Product Details	35	Admin Products	(admin)
15	Search	Search Products	36	Add / Edit Product	(admin)
16	Filter (sheet)	Product Filtering	37	Admin More	(admin)
17	Cart	Shopping Cart, Update Quantity, Remove	38	Admin Categories	(admin)
18	Wishlist	Wishlist / Favorite Products	39	Admin Customers	(admin)
19	Checkout (+ address form)	Checkout	40	Admin Customer Details	(admin)
20	KHQR Payment	(bonus: online payment)	41	Admin Messages	(admin)
21	Order Placed	Order Confirmation			

"Add to Cart" is a feature on Product List and Product Details (with an animation and an "Added to cart" sheet), so it has no separate screen. Screenshots of each screen are in `docs/screenshots/`.

---

10. Requirements traceability matrix

Each item of the brief, the requirements that cover it and where it is implemented.

Brief item	Requirement(s)	Implementation (app / API)	Status
UI/UX: modern, clean, smooth navigation	NFR-01, NFR-02	Figma file, design system, bottom navigation	Done
Splash Screen	FR-U01	`splash_screen.dart`	Done
Login	FR-U03	`login_screen.dart` / `POST /api/auth/login`	Done
Register	FR-U02	`register_screen.dart` / `POST /api/auth/register`	Done
Forgot Password	FR-U04	`forgot_password`, `verify_code`, `reset_password` screens / `/api/auth/forgot-password`, `verify-otp`, `reset-password`	Done
User Profile	FR-U05	`profile_screen.dart` / `GET /api/profile/summary`	Done
Edit Profile	FR-U06	`edit_profile_screen.dart` / `PUT /api/profile`, `POST /api/profile/avatar`	Done
Home Screen	FR-P01	`home_screen.dart` / `GET /api/products?featured=1`, `/api/series`	Done
Product Categories	FR-P02	`categories_screen.dart` / `GET /api/categories`	Done
Product List	FR-P03	`product_list_screen.dart` / `GET /api/products`	Done
Product Details	FR-P04	`product_details_screen.dart` / `GET`	Done

Brief item	Requirement(s)	Implementation (app / API)	Status
		`/api/products/<id>`	
Search Products	FR-P05	`search_screen.dart` / `GET /api/products?q=`, `/api/search/suggestions`	Done
Product Filtering	FR-P06	`filter_sheet.dart` / query parameters on `/api/products`	Done
Shopping Cart	FR-C04	`cart_screen.dart` / `GET /api/cart`	Done
Add to Cart	FR-C01	`POST /api/cart`	Done
Update Quantity	FR-C02	`quantity_chip.dart` / `PUT /api/cart/<id>`	Done
Remove Product from Cart	FR-C03	`DELETE /api/cart/<id>`, `DELETE /api/cart`	Done
Wishlist / Favorite Products	FR-C06	`wishlist_screen.dart` / `/api/wishlist/*`	Done
Checkout	FR-O01, FR-O02	`checkout_screen.dart` / `GET /api/checkout`, `POST /api/orders`	Done
Order Confirmation	FR-O03	`order_placed_screen.dart`	Done
Order History	FR-O05	`orders_screen.dart` / `GET /api/orders`	Done
Order Details	FR-O06, FR-O07	`order_details_screen.dart` / `GET /api/orders/<id>`	Done
Notifications	FR-A01	`notifications_screen.dart` / `/api/notifications/*`	Done
Settings	FR-A02	`settings_screen.dart` / `/api/profile/preferences`	Done
About Application	FR-A03	`about_screen.dart`, `legal_screen.dart`	Done
Contact Information	FR-A04	`contact_screen.dart` / `/api/contact`	Done
Backend core functions (7)	BE-01 ... BE-07	`backend/app/routes/*.py`	Done
API: JSON, errors and validation, docs	API-01 ... API-06	`app/errors.py`, `app/api_docs.py`, `/api/docs`, `docs/API.md`	Done
Database: ERD and schema	Section 8	`docs/database/`	Done
Flutter, Dart, state management	Section 3	Provider	Done
Git and GitHub	Section 3	github.com/Vanchha16/MA_Final	Done
Bonus: online payment	FR-O04	KHQR (Bakong)	Done
Bonus: multi-language	FR-A02, NFR-04	English / Khmer	Done
Bonus: dark mode	FR-A02	Light / dark / system	Done
Bonus: advanced search and filtering	FR-P05, FR-P06	Suggestions, multi-brand, price range, sort	Done
Bonus: product reviews and ratings	FR-R01 (below)	`reviews` table ready	*Planned*

---

11. Constraints, assumptions and future work

Constraints

- Built and demonstrated by a student team within one term; the backend runs on a local computer (WAMP MySQL on port 3306) for the demo.
- KHQR runs in **mock** mode for the demo (a demo QR and a "Simulate payment" button); setting `PAYMENT_PROVIDER=bakong` with a Bakong token makes it take real payments.
- Reset codes are printed in the server console in development (`MAIL_MODE=console`); real email needs SMTP settings.

Assumptions

- Customers have a smartphone with internet access and a Cambodian bank app for KHQR.
- Prices are in US dollars; delivery is within Phnom Penh.
- Product photos are the brands' official images, used for educational purposes only.

Future work

- **FR-R01 (Could):** customers rate and review products they received; product details show the average rating.
- Use the saved notification choices to filter which notifications are sent (v1.0 stores them but sends all order notifications).
- Khmer translation of the admin tools; push notifications (Firebase); deployment to a cloud server.